



Improving retention with long short-term memory networks

This chapter covers

- Adding deeper memory to recurrent neural nets
- Gating information inside neural nets
- Classifying and generating text
- Modeling language patterns

For all the benefits recurrent neural nets provide for modeling relationships, and therefore *possibly* causal relationships, in sequence data they suffer from one main deficiency: a token's *effect* is almost completely lost by the time two tokens have passed.¹ Any effect the first node has on the third node (two time steps after the first time step) will be thoroughly stepped on by new data introduced in the intervening time step. This is important to the basic structure of the net, but it prevents the common case in human language that the tokens may be deeply interrelated even when they're far apart in a sentence.

¹ Christopher Olah explains why: <https://colah.github.io/posts/2015-08-Understanding-LSTMs>.

Take this example:

The young woman went to the movies with her friends.

The subject “woman” immediately precedes its main verb “went.”² You learned in the previous chapters that both convolutional and recurrent nets would have no trouble learning from that relationship.

But in a similar sentence:

The young woman, having found a free ticket on the ground, went to the movies.

The noun and verb are no longer one time step apart in the sequence. A recurrent neural net is going to have difficulty picking up on the relationship between the subject “woman” and main verb “went” in this new, longer sentence. For this new sentence, a recurrent network would overemphasize the tie between the verb “having” and your subject “woman.” And your network would underemphasize the connection to “went,” the main verb of the predicate. You’ve lost the connection between the subject and verb of the sentence. The weights in a recurrent network decay too quickly in time as you roll through each sentence.

Your challenge is to build a network that can pick up on the same core *thought* in both sentences. What you need is a way to remember the past across the entire input sequence. A *long short-term memory* (LSTM) is just what you need.³

Modern versions of a long short-term memory network typically use a special neural network unit called a *gated recurrent unit* (GRU). A gated recurrent unit can maintain both long- and short-term memory efficiently, enabling an LSTM to process a long sentence or document more accurately.⁴ In fact, LSTMs work so well they have replaced recurrent neural networks in almost all applications involving time series, discrete sequences, and NLP.⁵

9.1 LSTM

LSTMs introduce the concept of a *state* for each layer in the recurrent network. The state acts as its *memory*. You can think of it as adding attributes to a class in object-oriented programming. The memory state’s attributes are updated with each training example.

² “Went” is the predicate (main verb) in this sentence. Find additional English grammar terminology at https://www.butte.edu/departments/cas/tipsheets/grammar/sentence_structure.html.

³ One of the first papers on LSTMs was by Hochreiter and Schmidhuber in 1997, “Long Short-Term Memory” (<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.676.4320&rep=rep1&type=pdf>).

⁴ “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” by Kyunghyun Cho et al, 2014: <https://arxiv.org/pdf/1406.1078.pdf>.

⁵ Christopher Olah’s blog post explains why this is: <https://colah.github.io/posts/2015-08-Understanding-LSTMs>.

In LSTMs, the rules that govern the information stored in the state (memory) are trained neural nets themselves—therein lies the magic. They can be trained to learn what to remember, while at the same time the rest of the recurrent net learns to predict the target label! With the introduction of a memory and state, you can begin to learn dependencies that stretch not just one or two tokens away, but across the entirety of each data sample. With those long-term dependencies in hand, you can start to see beyond the words themselves and into something deeper about language.

With LSTMs, patterns that humans take for granted and process on a subconscious level begin to be available to your model. And with those patterns, you can not only more accurately predict sample classifications, but you can start to generate novel text using those patterns. The state of the art in this field is still far from perfect, but the results you'll see, even in your toy examples, are striking.

So how does this thing work (see figure 9.1)?

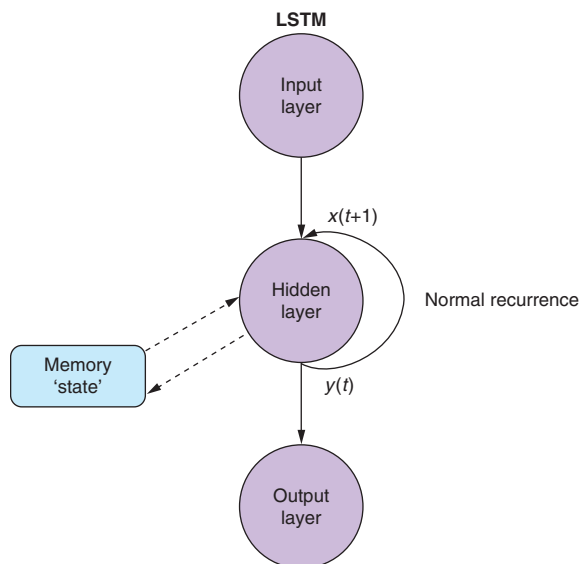


Figure 9.1 LSTM network and its memory

The memory state is affected by the input and also affects the layer output just as in a normal recurrent net. But that memory state persists across all the time steps of the time series (your sentence or document). So each input can have an effect on the memory state as well as an effect on the hidden layer output. The magic of the memory state is that it *learns* what to remember at the same time that it learns to reproduce the output, using standard backpropagation! So what does this look like?

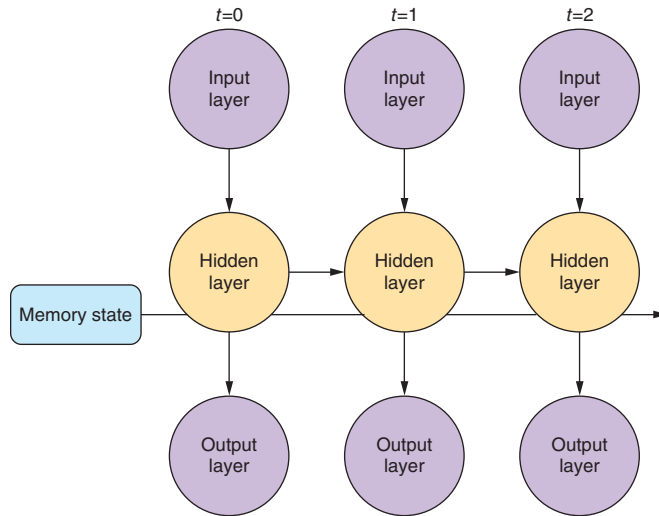


Figure 9.2 Unrolled LSTM network and its memory

First, let's unroll a standard recurrent neural net and add your memory unit. Figure 9.2 looks similar to a normal recurrent neural net. However, in addition to the activation output feeding into the next time-step version of the layer, you add a memory state that also passes through time steps of the network. At each time-step iteration, the hidden recurrent unit has access to the memory unit. The addition of this memory unit, and the mechanisms that interact with it, make this quite a bit different from a traditional neural network layer. However, you may like to know that it's possible to design a set of traditional recurrent neural network layers (a computational graph) that accomplishes all the computations that exist within an LSTM layer. An LSTM layer is just a highly specialized recurrent neural network.

TIP In much of the literature,⁶ the “Memory State” block shown in figure 9.2 is referred to as an LSTM *cell* rather than an LSTM *neuron*, because it contains two additional neurons or gates just like a silicon computer memory *cell*.⁷ When an LSTM memory *cell* is combined with a sigmoid activation function to output a value to the next LSTM *cell*, this structure, containing multiple interacting elements, is referred to as an LSTM *unit*. Multiple LSTM *units* are combined to form an LSTM *layer*. The horizontal line running across the unrolled recurrent neuron in figure 9.2 is the signal holding the memory or state. It becomes a vector with a dimension for each LSTM *cell* as the sequence of tokens is passed into a multi-unit LSTM *layer*.

⁶ A good recent example of LSTM terminology usage is Alex Graves' 2012 Thesis “Supervised Sequence Labeling with Recurrent Neural Networks”: <https://mediatum.ub.tum.de/doc/673554/file.pdf>.

⁷ See the Wikipedia article “Memory cell” ([https://en.wikipedia.org/wiki/Memory_cell_\(computing\)](https://en.wikipedia.org/wiki/Memory_cell_(computing))).

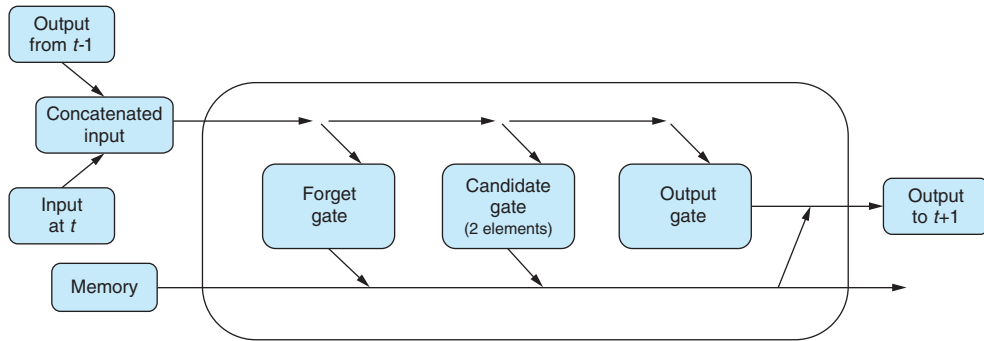


Figure 9.3 LSTM layer at time step t

So let's take a closer look at one of these cells. Instead of being a series of weights on the input and an activation function on those weights, each cell is now somewhat more complicated. As before, the input to the layer (or cell) is a combination of the input sample and output from the previous time step. As information flows into the cell instead of a vector of weights, it's now greeted by three gates: a forget gate, an input/candidate gate, and an output gate (see figure 9.3).

Each of these gates is a feed forward network layer composed of a series of weights that the network will learn, plus an activation function. Technically one of the gates is composed of two feed forward paths, so there will be four sets of weights to learn in this layer. The weights and activations will aim to *allow* information to flow through the cell in different amounts, all the way through to the cell's state (or memory).

Before getting too deep in the weeds, let's look at this in Python, using the example from the previous chapter with the SimpleRNN layer swapped out for an LSTM. You can use the same vectorized, padded/truncated processed data from the last chapter, `x_train`, `y_train`, `x_test`, and `y_test`. See the following listing.

Listing 9.1 LSTM layer in Keras

```
>>> maxlen = 400
>>> batch_size = 32
>>> embedding_dims = 300
>>> epochs = 2
>>> from keras.models import Sequential
>>> from keras.layers import Dense, Dropout, Flatten, LSTM
>>> num_neurons = 50
>>> model = Sequential()
>>> model.add(LSTM(num_neurons, return_sequences=True,
...               input_shape=(maxlen, embedding_dims)))
>>> model.add(Dropout(.2))
>>> model.add(Flatten())
>>> model.add(Dense(1, activation='sigmoid'))
>>> model.compile('rmsprop', 'binary_crossentropy', metrics=['accuracy'])
>>> print(model.summary())
```

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 400, 50)	70200
dropout_1 (Dropout)	(None, 400, 50)	0
flatten_1 (Flatten)	(None, 20000)	0
dense_1 (Dense)	(None, 1)	20001
Total params: 90,201.0		
Trainable params: 90,201.0		
Non-trainable params: 0.0		

One import and one line of Keras code changed. But a great deal more is going on under the surface. From the summary, you can see you have many more parameters to train than you did in the SimpleRNN from last chapter for the same number of neurons (50). Recall the simple RNN had the following weights:

- 300 (one for each element of the input vector)
- 1 (one for the bias term)
- 50 (one for each neuron's output from the previous time step)

For a total of 351 per neuron.

$351 * 50 = 17,550$ for the layer

The cells have three gates (a total of four neurons):

$17,550 * 4 = 70,200$

But what is the memory? The memory is going to be represented by a vector that is the same number of elements as neurons in the cell. Your example has a relatively simple 50 neurons, so the memory unit will be a vector of floats that is 50 elements long.

Now what are these gates? Let's follow the first sample on its journey through the net and get an idea (see figure 9.4).

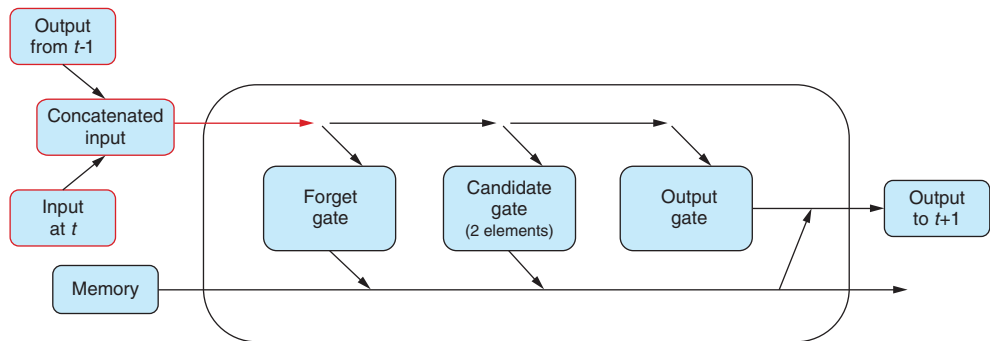


Figure 9.4 LSTM layer inputs

The “journey” through the cell isn’t a single road; it has branches, and you’ll follow each for a while then back up, progress, branch, and finally come back together for the grand finale of the cell’s output.

You take the first token from the first sample and pass its 300-element vector representation into the first LSTM cell. On the way into the cell, the vector representation of the data is concatenated with the vector output from the previous time step (which is a 0 vector in the first time step). In this example, you’ll have a vector that is $300 + 50$ elements long. Sometimes you’ll see a 1 appended to the vector—this corresponds to the bias term. Because the bias term always multiplies its associated weight by a value of one before passing to the activation function, that input is occasionally omitted from the input vector representation, to keep the diagrams more digestible.

At the first fork in the road, you hand off a copy of the combined input vector to the ominous sounding *forget gate* (see figure 9.5). The forget gate’s goal is to learn, based on a given input, how much of the cell’s memory you want to erase. Whoa, wait a minute. You just got this memory thing plugged in and the first thing you want to do is start erasing things? Sheesh.

The idea behind wanting to forget is as important as wanting to remember. As a human reader, when you pick up certain bits of information from text—say whether the noun is singular or plural—you want to retain that information so that later in the sentence you can recognize the right verb conjugation or adjective form to match with it. In romance languages, you’d have to recognize a noun’s gender, too, and use that later in a sentence as well. But an input sequence can easily switch from one noun to another, because an input sequence can be composed of multiple phrases, sentences,

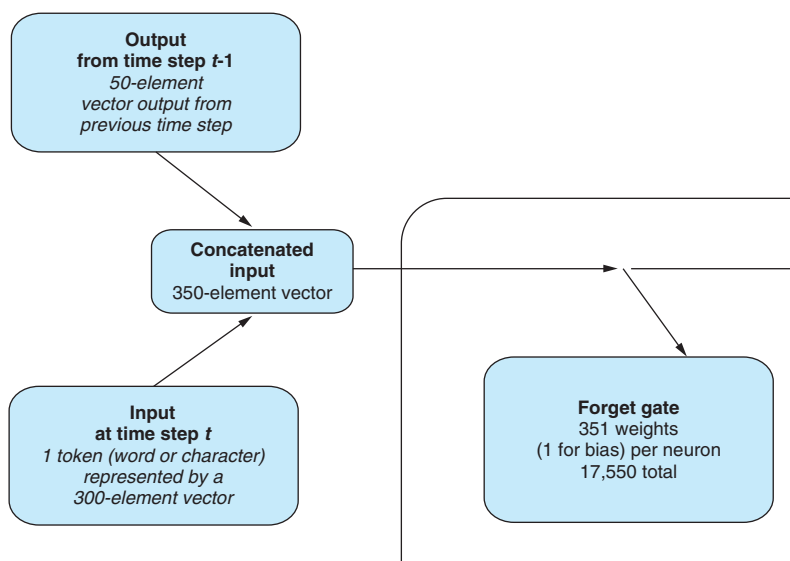


Figure 9.5 First stop—the forget gate

or even documents. As new thoughts are expressed in later statements, the fact that the noun is plural may not be at all relevant to later unrelated text.

A thinker sees his own actions as experiments and questions—as attempts to find out something. Success and failure are for him answers above all.

Friedrich Nietzsche

In this quote, the verb “see” is conjugated to fit with the noun “thinker.” The next active verb you come across is “to be” in the second sentence. At that point “be” is conjugated into “are” to agree with “Success and failure.” If you were to conjugate it to match the first noun you came across, “thinker,” you would use the wrong verb form, “is” instead. So an LSTM must model not only long-term dependencies within a sequence, but just as crucially, also forget long-term dependencies as new ones arise. This is what forgetting gates are for, making room for the *relevant* memories in your memory cells.

The network isn’t working with these kinds of explicit representations. Your network is trying to find a set of weights to multiply by the inputs from the sequence of tokens so that the memory cell and the output are both updated in a way that minimizes the error. It’s amazing that they work at all. And they work very well indeed. But enough marveling; back to forgetting.

The forget gate itself (shown in figure 9.6) is just a feed forward network. It consists of n neurons each with $m + n + 1$ weights. So your example forget gate has 50 neurons each with 351 ($300 + 50 + 1$) weights. The activation function for a forget gate is the sigmoid function, because you want the output for each neuron in the gate to be between 0 and 1.

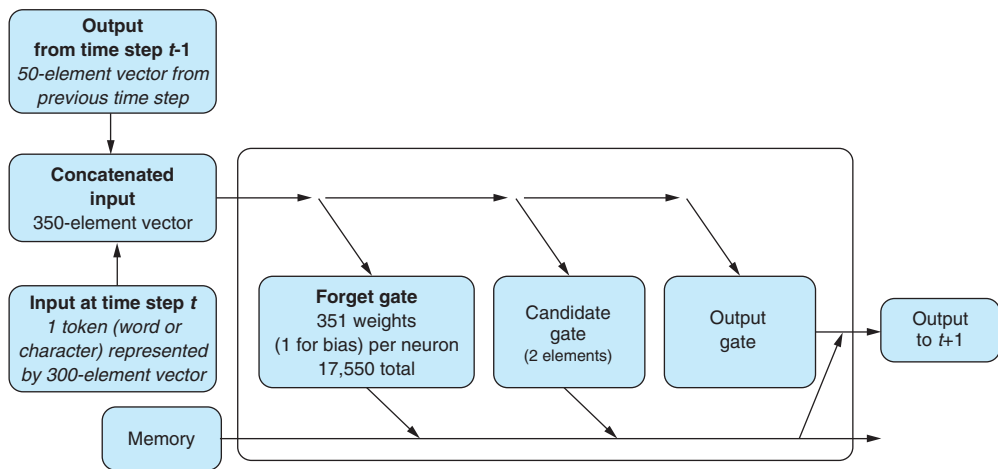


Figure 9.6 Forget gate

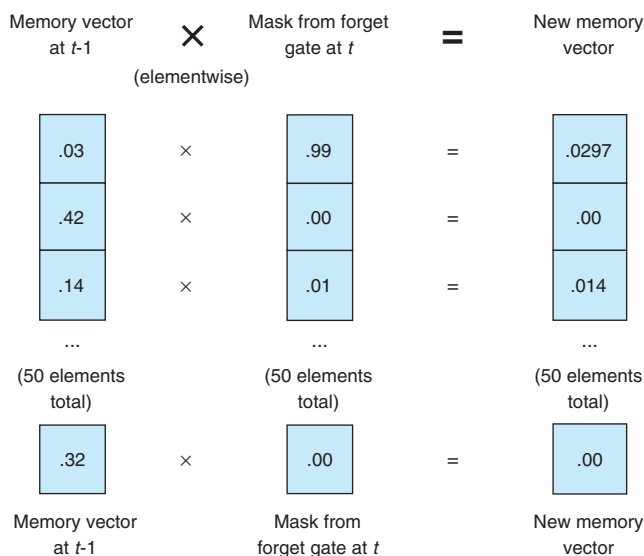


Figure 9.7 Forget gate application

The output vector of the forget gate is then a mask of sorts, albeit a porous one, that erases elements of the memory vector. As the forget gate outputs values closer to 1, more of the memory’s knowledge in the associated element is retained for that time step; the closer it is to 0 the more of that memory value is erased (see figure 9.7).

Actively forgetting things, check. You better learn how to remember something new, or this is going to go south pretty quickly. Just like in the forget gate, you use a small network to learn how much to augment the memory based on two things: the input so far and the output from the last time step. This is what happens in the next gate you branch into: the *candidate gate*.

The candidate gate has two separate neurons inside it that do two things:

- 1 Decide which input vector elements are worth remembering (similar to the mask in the forget gate)
- 2 Route the remembered input elements to the right memory “slot”

The first part of a candidate gate is a neuron with a sigmoid activation function whose goal is to learn which input values of the memory vector to update. This neuron closely resembles the mask in the forget gate.

The second part of this gate determines what values you’re going to update the memory with. This second part has a *tanh* activation function that forces the output value to range between -1 and 1. The output of these two vectors are multiplied together elementwise. The resulting vector from this multiplication is then added, again elementwise, to the memory register, thus remembering the new details (see figure 9.8).

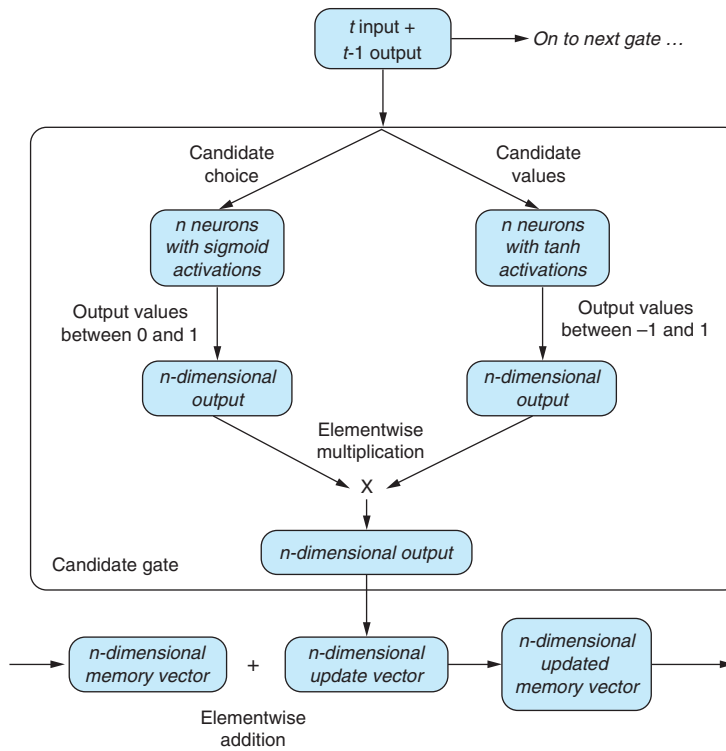


Figure 9.8 Candidate gate

This gate is learning simultaneously which values to extract and the magnitude of those particular values. The mask and magnitude become what's added to the memory state. As in the forget gate, the candidate gate is learning to mask off the inappropriate information before adding it to the cell's memory.

So old, hopefully irrelevant things are forgotten, and new things are remembered. Then you arrive at the last gate of the cell: the *output gate*.

Up until this point in the journey through the cell, you've only written to the cell's memory. Now it's finally time to get some use out of this structure. The output gate takes the input (remember this is still the concatenation of the input at time step t and the output of the cell at time step $t-1$) and passes it into the output gate.

The concatenated input is passed into the weights of the n neurons, and then you apply a sigmoid activation function to output an n -dimensional vector of floats, just like the output of a SimpleRNN. But instead of handing that information out through the cell wall, you pause.

The memory structure you've built up is now primed, and it gets to weigh in on what you *should* output. This judgment is achieved by using the memory to create one last mask. This mask is a kind of gate as well, but you refrain from using that term

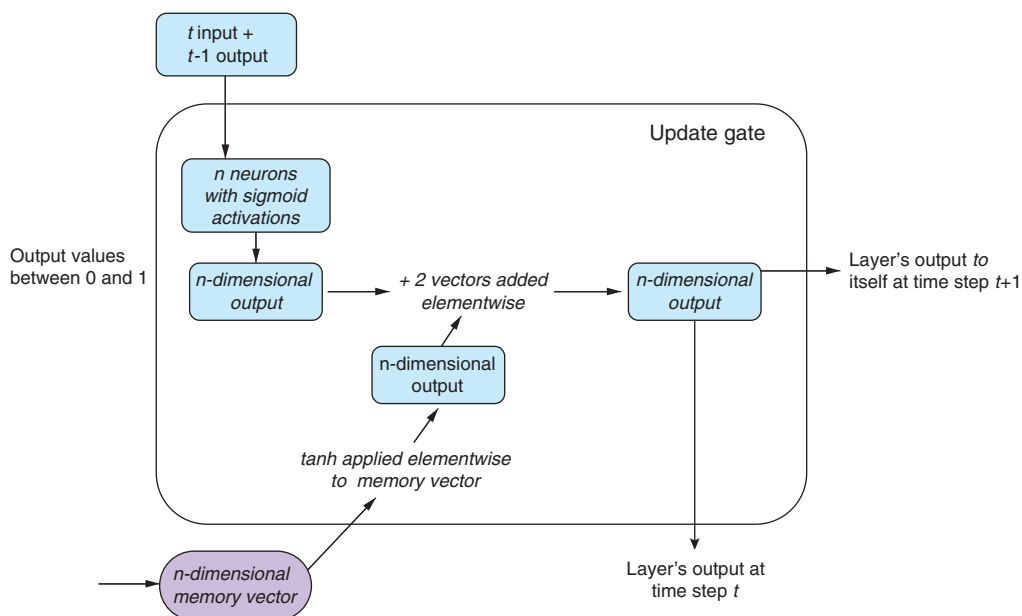


Figure 9.9 Update/output gate

because this mask doesn't have any learned parameters, which differentiates it from the three previous gates described.

The mask created from the memory is the memory state with a $\tanh$ function applied elementwise, which gives an n -dimensional vector of floats between -1 and 1. That mask vector is then multiplied elementwise with the raw vector computed in the output gate's first step. The resulting n -dimensional vector is finally passed out of the cell as the cell's official output at time step t (see figure 9.9).

TIP Remember that the output from an LSTM cell is like the output from a simple recurrent neural network layer. It's passed out of the cell as the layer output (at time step t) and to itself as part of the input to time step $t+1$.

Thereby the memory of the cell gets the last word on what's important to output at time step t , given what was input at time step t and output at $t-1$, and all the details it has gleaned up to this point in the input sequence.

9.1.1 Backpropagation through time

How does this thing learn then? Backpropagation—as with any other neural net. For a moment, let's step back and look at the problem you're trying to solve with this new complexity. A vanilla RNN is susceptible to a vanishing gradient because the derivative at any given time step is a factor of the weights themselves, so as you step back in time coalescing the various deltas, after a few iterations, the weights (and the learning rate)

may shrink the gradient away to 0. The update to the weights at the end of the back-propagation (which would equate to the beginning of the sequence) are either minuscule or effectively 0. A similar problem occurs when the weights are somewhat large: the gradient *explodes* and grows disproportionately to the network.

An LSTM avoids this problem via the memory state itself. The neurons in each of the gates are updated via derivatives of the functions they fed, namely those that update the memory state on the forward pass. So at any given time step, as the normal chain rule is applied backwards to the forward propagation, the updates to the neurons are dependent on only the memory state at that time step and the previous one. This way, the error of the entire function is kept “nearer” to the neurons for each time step. This is known as the *error carousel*.

IN PRACTICE

How does this work in practice then? Exactly like the simple RNN from the last chapter. All you’ve changed is the inner workings of the black box that’s a recurrent layer in the network. So you can just swap out the Keras SimpleRNN layer for the Keras LSTM layer, and all the other pieces of your classifier will stay the same.

You’ll use the same dataset, prepped the same way: tokenize the text and embed those using Word2vec. Then you’ll pad/truncate the sequences again to 400 tokens each using the functions you defined in the previous chapters. See the following listing.

Listing 9.2 Load and prepare the IMDB data

```
>>> import numpy as np

>>> dataset = pre_process_data('./aclimdb/train')
>>> vectorized_data = tokenize_and_vectorize(dataset)
>>> expected = collect_expected(dataset)
>>> split_point = int(len(vectorized_data) * .8)

>>> x_train = vectorized_data[:split_point]
>>> y_train = expected[:split_point]
>>> x_test = vectorized_data[split_point:]
>>> y_test = expected[split_point:]

>>> maxlen = 400
>>> batch_size = 32
>>> embedding_dims = 300
>>> epochs = 2

>>> x_train = pad_trunc(x_train, maxlen)
>>> x_test = pad_trunc(x_test, maxlen)
>>> x_train = np.reshape(x_train,
... (len(x_train), maxlen, embedding_dims))
>>> y_train = np.array(y_train)
>>> x_test = np.reshape(x_test, (len(x_test), maxlen, embedding_dims))
>>> y_test = np.array(y_test)
```

← Gather the data and prep it.

← Split the data into training and testing sets.

← Declare the hyperparameters.

← Number of samples to show the net before backpropagating the error and updating the weights.

← Length of the token vectors we will create for passing into the Convnet

← Further prep the data by making each point of uniform length.

← Reshape into a numpy data structure.